A Symbolic Constraint Satisfaction Solver for Zebra Logic Puzzles

1. Introduction

Logic grid puzzles (Zebra puzzles) are a classical benchmark for symbolic reasoning in artificial intelligence and can be naturally formulated as Constraint Satisfaction Problems (CSPs). In the context of the **AI Connect** project, this work presents a purely symbolic CSP solver designed to solve Zebra-style logic puzzles of varying sizes and structures.

The solver focuses on correctness, efficiency, and generalization. Rather than relying on learning-based methods, it combines careful CSP modeling with established search heuristics and constraint propagation techniques.

---

2. Problem Formulation

Each Zebra puzzle is modeled as a CSP defined by the tuple *(Variables, Domains, Constraints)*.

Variables

Each attribute–value pair is represented as a CSP variable. For example, values such as `Name_Alice`, `Color_Red`, or `Pet_Dog` are modeled as individual variables. The value assigned to each variable corresponds to a house index.

Domains

All variables initially share the same domain `{1, …, N}`, where *N* is the number of houses in the puzzle. Domains are progressively reduced through constraint propagation and search.

Constraints

Constraints are derived from both the structural rules of Zebra puzzles and the parsed natural-language clues:

- **AllDifferent constraints** ensure that values within the same attribute category (e.g., names or colors) occupy distinct houses.
- **Unary constraints** encode fixed assignments or exclusions.
- **Binary constraints** represent relations such as equality, inequality, adjacency, ordering (left/right), and distance-based relations.

This formulation allows puzzles with different numbers of houses, attributes, and clue types to be handled in a uniform manner.

---

3. System Architecture

The system is organized into four main components:

1. **Data Parsing Module**
   Reads puzzles from the ZebraLogicBench dataset and converts them into a structured CSP representation, including variables, domains, and constraints.

2. **CSP Solver Core**
   Implements a backtracking-based CSP solver enhanced with inference, heuristics, and propagation mechanisms.

3. **Trace Generator**
   Records solver decisions at each search step, including variable selection, value assignment, domain pruning, and backtracking events.

4. **Evaluation Module**
   Executes the solver on validation and test splits and computes accuracy, efficiency metrics, and summary statistics.

The modular design supports debugging, analysis, and future extensions without affecting the solver core.

---

4. Solving Approach

The solver is based on a depth-first backtracking CSP framework augmented with standard heuristics and propagation mechanisms to reduce the search space and detect inconsistencies early.

Search strategy. The solver incrementally builds partial assignments and backtracks whenever a constraint violation or empty domain is encountered.

Variable ordering. The Minimum Remaining Values (MRV) heuristic selects the most constrained unassigned variable. Ties are broken using a degree heuristic that prefers variables participating in many constraints.

Value ordering. Domain values are tried using a Least Constraining Value (LCV) heuristic, prioritizing assignments that eliminate the fewest options for neighboring variables. Values that cause immediate domain wipeouts are avoided when possible.

Constraint propagation. Forward checking prunes inconsistent values after each assignment, while arc consistency (AC-3) removes unsupported values across all binary constraints. Propagation is applied both as preprocessing and during search.

Backtracking optimization. Conflict-directed backjumping and nogood learning prevent redundant exploration of failing branches.

Efficiency considerations. Constraint checks are limited to relevant neighbors, arc-consistency queues avoid duplicates, and watched-literal-style bookkeeping reduces unnecessary re-evaluation.

---

5. Trace Generation

The solver optionally records detailed execution traces for analysis. Each trace entry includes:

- Selected variable and assigned value
- Domain size statistics
- Number of remaining values
- Constraint counts
- Action type (assignment, pruning, backtracking, propagation)

Traces are not used to guide the solver itself but provide insight into solver behavior and efficiency and support post-hoc analysis.

---

6. Experimental Setup

- **Dataset:** ZebraLogicBench (approximately 1,000 puzzles)
- **Evaluation Splits:** Official validation and held-out test sets
- **Execution Environment:** CPU-only, single-threaded Python execution
- **Solver Type:** Purely symbolic (no learning-based components)
- **Team Size:** 7 members

All clue types present in the dataset, including positional, relational, and distance-based constraints, are supported by the solver.

---

7. Results

Quantitative Performance

Metric	Value
Validation Accuracy	98%
Average Search Steps	18.99
Average Runtime per Puzzle	2.43 ms

The solver achieves a strong balance between correctness and efficiency, solving the vast majority of puzzles with relatively shallow search.

---

8. Discussion

The results demonstrate that a carefully engineered symbolic CSP solver can perform competitively on large and diverse logic puzzle benchmarks. The combination of informed variable and value ordering, aggressive constraint propagation, and optimized backtracking significantly reduces search effort while maintaining generality across puzzle sizes.

Most remaining failures can be attributed to limitations in natural-language clue parsing rather than deficiencies in the CSP solving process itself.

---

9. Conclusion

This project demonstrates that classical symbolic reasoning techniques remain effective for structured logical reasoning tasks. By combining robust CSP modeling with established heuristics and propagation methods, the presented solver achieves high accuracy and efficiency on ZebraLogicBench. The modular design and trace infrastructure provide a solid foundation for future extensions, including hybrid symbolic–learning approaches.